Implementing Interactive Theorem Prover from Scratch

HIDEAKI TAKAHASHI, Columbia University, USA

TBD

1 PROPOSITIONS AS TYPES

A theorem prover has to decide whether a proof is correct. A proof written in English cannot be checked by a computer, so we need a way to write proofs as data, together with a precise rule that tells correct proofs from incorrect ones. The idea that makes this possible is known as *propositions as types*, or the *Curry–Howard correspondence* [**?**]. It says that proofs can be written as programs, and that checking a proof is the same as type checking a program. This section builds up the idea in four steps, starting from what a proof is.

1.1 Step 1: A Proof Is Something You Can Hand Over

Think of a proof as the evidence you would hand to a skeptical reader to convince them that a statement holds. What evidence is needed depends on the shape of the statement.

- To convince someone of "A and B", you hand over two things: evidence for A and evidence for B.
- To convince someone of "A implies B", you cannot hand over evidence for A, because A might not hold. Instead you hand over a *method*: whenever someone gives you evidence for A, the method turns it into evidence for B.
- To convince someone of "for every number n, $P(n)$", you hand over a method that, given any n, produces evidence for $P(n)$.
- For "false", there is nothing to hand over. That is what makes it false.

The important point is that proofs are *objects*: they can be built, combined, and transformed into other proofs.

1.2 Step 2: A Method Is a Function

Start with something familiar. A function such as *length* turns any string into a number, and we record this by writing $\mathit{length} : \mathsf{String} \to \mathsf{Nat}$. The arrow is read "the type of functions from strings to numbers".

Now take two everyday statements:

$$R = \text{"it is raining"} \qquad W = \text{"the street is wet"}.$$

By Step 1, a proof of "R implies W" is a method that turns any evidence that it is raining into evidence that the street is wet. If we also use the letter R for the collection of all evidence for R, and likewise for W, then that method is simply a function from R to W:

$$\mathit{proof} : R \to W.$$

So the arrow of logic ("implies") and the arrow of function types are *the same arrow*. A proof of an implication is a function.

Some proofs, written as functions. We write $x \mapsto e$ for the function that takes an input x and returns e. Here P, A, B, C stand for arbitrary statements.

Author's address: Hideaki Takahashi, ht2673@columbia.edu, Columbia University, USA.

Statement	Proof	In words
$P \to P$	$h \mapsto h$	Given evidence h for P, hand h back.
$A \to (B \to A)$	$a \mapsto (b \mapsto a)$	Given a for A, then b for B, ignore b and return a.
$(A \to B) \to (B \to C) \to (A \to C)$	$f \mapsto g \mapsto a \mapsto g(f(a))$	Turn a into evidence for B with f, then into evidence for C with g.

The second row shows how to handle two premises: a function that takes a and returns *another function* that takes b. The third row is the familiar fact that implication is transitive, and its proof is function composition.

A statement with no proof. Now try to prove "P implies Q" for two unrelated statements P and Q. We would need a function $p \mapsto ?$ where ? is evidence for Q. The only thing we have is p, which is evidence about P, not Q. There is nothing to put in place of ?. No function exists, just as no proof exists.

In most programming languages a function can still *claim* to return evidence for Q by crashing or by running forever. Such a function never actually delivers anything, so it is not evidence. A type theory meant for proofs must forbid these functions, and the system in this paper does: every well-typed term finishes computing (§11).

1.3 Step 3: A Proposition Is the Type of Its Proofs

Step 2 suggests a precise definition. We identify a proposition with the *type of its proofs*:

> *a proof of A is a term t with $t : A$, and A is true when the type A has at least one element.*

This is a change of viewpoint from everyday programming, where a proposition is usually a `bool`. A `bool` only says *whether* something holds. A type says what counts as *evidence* that it holds, and its elements are those pieces of evidence. The kernel never has to decide whether a statement is true, which is impossible in general. It only has to check that a given piece of evidence has the right type, which is easy.

1.4 Step 4: Quantifying Over Propositions

In Step 2, $h \mapsto h$ proves $P \to P$ for whatever statement P is. But "for whatever P is" was only said in the surrounding English; it was not part of the statement. To make "for every proposition P" part of the statement itself, we let P be an input too. The proof first receives *which* proposition P is meant, and then evidence h for it:

$$P \mapsto (h \mapsto h).$$

The rest of this paper uses the standard notation for such functions. We write $\lambda(x{:}A).\, e$ for $x \mapsto e$, recording the type A of the input, and Π for the type of such functions. In this notation, the statement "for every proposition P, P implies P" is

$$\Pi(P{:}\mathrm{Sort}\ 0).\, P \to P,$$

and its proof is

$$\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h.$$

Sort 0 is the type whose elements are propositions, so $P : \mathrm{Sort}\ 0$ says "P is a proposition". The following sections define each of these pieces precisely.

1.5 Summary

Table 1 collects the correspondence. The right column is what a kernel implements; the left column is what it means. In particular, the rules of logic become typing rules: using a hypothesis is a variable, proving an implication by assuming its premise is a λ, and modus ponens is function application (§7).

Table 1. The Curry–Howard correspondence.

Logic	**Programs**
proposition A	type A
proof of A	term t with $t : A$
A is true	type A has an element
A implies B	function type $A \to B$
A and B	pair type $A \times B$
false	a type with no elements
for every x in A, B holds	dependent function type $\Pi(x{:}A).\, B$
using a hypothesis	variable
proving $A \to B$ by assuming A	λ-abstraction
modus ponens	function application
checking a proof	type checking

Pairs do not appear in the core theory of this paper; they are listed only to connect with Step 1.

2 SYNTAX

We work with a small dependent type theory with five kinds of terms.

Definition 1 (Terms). *Let x, y, z range over an infinite set of variable names and u, v over natural numbers. Terms are given by the grammar*

$$s, t, A, B \ ::= \ \mathsf{Sort}\ u \ \mid \ x \ \mid \ \Pi(x{:}A).\, B \ \mid \ \lambda(x{:}A).\, t \ \mid \ t\ s$$

- $\mathsf{Sort}\ u$ is a *universe*: a type whose elements are themselves types (§6).
- x is a variable.
- $\Pi(x{:}A).\, B$ is the *type* of functions that take an argument x of type A and return a value of type B, where B may mention x.
- $\lambda(x{:}A).\, t$ is a function *value*: it takes x of type A and returns t.
- $t\ s$ applies the function t to the argument s. Application associates to the left: $f\ a\ b$ means $(f\ a)\ b$.

When x does not occur in B, we write $A \to B$ for $\Pi(x{:}A).\, B$. The arrow associates to the right: $A \to B \to C$ means $A \to (B \to C)$. So a single constructor Π covers both implication and universal quantification. This is the standard design in dependent type theory: Lean, Coq, and Agda all have a single Π (or $\forall$) in their kernels and treat $A \to B$ as notation.

The grammar has no separate category for types. A type is simply a term whose type is a universe. This is necessary because, with dependent types, values appear inside types (as in Vec n) and types are passed around as values (as P is in the example above). The price is that the grammar alone cannot rule out nonsense such as using a λ-abstraction as a type; the type checker has to do it.

Pi and Lambda. Π and λ have the same shape but live at different levels. $\lambda(x{:}A).\,t$ is a function, while $\Pi(x{:}A).\,B$ is the type of such functions. In the body position, λ holds the value that is returned and Π holds the type of that value.

$$\underbrace{\lambda(P{:}\mathrm{Sort}\ 0).\,\lambda(h{:}P).\,h}_{\text{a proof}} \;:\; \underbrace{\Pi(P{:}\mathrm{Sort}\ 0).\,P \to P}_{\text{a proposition}} \;:\; \underbrace{\mathrm{Sort}\ 1}_{\text{a universe}}$$

Implementation. In Rust the grammar becomes one recursive enum. Subterms are boxed because a recursive type must have a finite size.

```
#[derive(Clone, Debug, PartialEq, Eq)]
enum Term {
    Sort(u64),
    Var(String),
    Pi(String, Box<Term>, Box<Term>),
    Lam(String, Box<Term>, Box<Term>),
    App(Box<Term>, Box<Term>),
}
```

3 FREE VARIABLES AND SUBSTITUTION

Π and λ *bind* their variable: in $\lambda(x{:}A).\,t$, occurrences of x inside t refer to the argument. The binder's scope is the body only; x is not bound in the annotation A.

Definition 2 (Free variables).

$$\begin{aligned}
\mathrm{FV}(\mathrm{Sort}\ u) &= \emptyset & \mathrm{FV}(\Pi(x{:}A).\,B) &= \mathrm{FV}(A) \cup (\mathrm{FV}(B) \setminus \{x\}) \\
\mathrm{FV}(x) &= \{x\} & \mathrm{FV}(\lambda(x{:}A).\,t) &= \mathrm{FV}(A) \cup (\mathrm{FV}(t) \setminus \{x\}) \\
\mathrm{FV}(t\ s) &= \mathrm{FV}(t) \cup \mathrm{FV}(s)
\end{aligned}$$

The typing rule for application (§7) needs to replace a variable with a term. We write $t[x \mapsto s]$ for the result of replacing the free occurrences of x in t with s. The only subtle case is a binder. Consider

$$(\lambda(y{:}A).\,x)[x \mapsto y].$$

The body says "ignore the argument and return the outer x". Replacing x by y naively gives $\lambda(y{:}A).\,y$, the identity function, which means something else: the free y has been *captured* by the binder. Substitution must avoid this by renaming the binder first.

Definition 3 (Capture-avoiding substitution). *For* $\square \in \{\Pi, \lambda\}$*:*

$$\begin{aligned}
x[x \mapsto s] &= s \\
y[x \mapsto s] &= y && (y \neq x) \\
\mathrm{Sort}\ u[x \mapsto s] &= \mathrm{Sort}\ u \\
(t_1\ t_2)[x \mapsto s] &= t_1[x \mapsto s]\ \ t_2[x \mapsto s] \\
(\square(x{:}A).\,B)[x \mapsto s] &= \square(x{:}A[x \mapsto s]).\,B && (x \text{ is shadowed}) \\
(\square(y{:}A).\,B)[x \mapsto s] &= \square(y{:}A[x \mapsto s]).\,B[x \mapsto s] && (y \neq x,\ y \notin \mathrm{FV}(s)) \\
(\square(y{:}A).\,B)[x \mapsto s] &= \square(z{:}A[x \mapsto s]).\,(B[y \mapsto z])[x \mapsto s] && (y \neq x,\ y \in \mathrm{FV}(s),\ z \text{ fresh})
\end{aligned}$$

Here "z fresh" means $z \notin \mathrm{FV}(s) \cup \mathrm{FV}(B) \cup \{x\}$.

4 ALPHA-EQUIVALENCE

The names of bound variables carry no meaning. $\lambda(x{:}A).\, x$ and $\lambda(y{:}A).\, y$ are the same function, and $\Pi(P{:}\text{Sort } 0).\, P \to P$ and $\Pi(Q{:}\text{Sort } 0).\, Q \to Q$ are the same proposition. Terms that differ only in the names of bound variables are called *α-equivalent*.

Definition 4 (α-equivalence). *$=_\alpha$ is the smallest congruence on terms that contains, for $\square \in \{\Pi, \lambda\}$,*

$$\square(x{:}A).\, B \;=_\alpha\; \square(y{:}A).\, B[x \mapsto y] \qquad \textit{whenever } y \notin \mathrm{FV}(B).$$

This definition is easy to state but not directly executable, since it does not say which names to choose. An implementation instead walks both terms in lockstep and remembers, for each side, the list of binders passed so far. A bound variable is then identified by *which* binder it refers to, not by its name. We write $E_1; E_2 \vdash s =_\alpha t$, where E_1 and E_2 are lists of names. Let $idx(E, x)$ be the position of the last occurrence of x in E, counted from the right starting at 0, or undefined if $x \notin E$. Taking the *last* occurrence handles shadowing: in $\lambda(x{:}A).\, \lambda(x{:}B).\, x$ the body refers to the inner x.

$$\frac{}{E_1; E_2 \vdash \text{Sort } u =_\alpha \text{Sort } u}\ (\alpha\text{-Sort}) \qquad \frac{idx(E_1, x) = idx(E_2, y) = i}{E_1; E_2 \vdash x =_\alpha y}\ (\alpha\text{-Bound}) \qquad \frac{x \notin E_1 \qquad x \notin E_2}{E_1; E_2 \vdash x =_\alpha x}\ (\alpha\text{-Free})$$

$$\frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash B =_\alpha B'}{E_1; E_2 \vdash \Pi(x{:}A).\, B =_\alpha \Pi(y{:}A').\, B'}\ (\alpha\text{-Pi}) \qquad \frac{E_1; E_2 \vdash A =_\alpha A' \qquad E_1, x; E_2, y \vdash t =_\alpha t'}{E_1; E_2 \vdash \lambda(x{:}A).\, t =_\alpha \lambda(y{:}A').\, t'}\ (\alpha\text{-Lam})$$

$$\frac{E_1; E_2 \vdash t_1 =_\alpha t_1' \qquad E_1; E_2 \vdash t_2 =_\alpha t_2'}{E_1; E_2 \vdash t_1\ t_2 =_\alpha t_1'\ t_2'}\ (\alpha\text{-App})$$

Fig. 1. Algorithmic α-equivalence. Two closed or open terms are α-equivalent iff $\cdot\,; \cdot \vdash s =_\alpha t$.

Free variables are compared by name (α-Free), and bound variables by position (α-Bound). A variable that is bound on one side and free on the other is never equal to anything, which is why there is no rule for that case. Because both sides descend through binders together, E_1 and E_2 always have the same length. So comparing positions counted from the right is the same as comparing positions counted from the left, and an implementation may store each list as a map from names to binding depths, provided it restores the previous entry when it leaves a binder.

Using positions instead of names everywhere gives the *de Bruijn representation* [**?**], in which α-equivalent terms are syntactically identical. We keep names in this text because they are easier to read.

5 REDUCTION AND CONVERSION

Applying a λ-abstraction computes by substitution:

$$(\lambda(x{:}A).\, t)\ s \;\to_\beta\; t[x \mapsto s].$$

A subterm of the form $(\lambda(x{:}A).\, t)\ s$ is called a *β-redex*, and we also allow this step anywhere inside a term. A term with no β-redex is in *normal form*.

Computation can make two different-looking types equal. If $Id = \lambda(X{:}\text{Sort } 0).\, X$, then a proof of $Id\ P$ should also be a proof of P, since $Id\ P \to_\beta P$. This is captured by *definitional equality* (or *conversion*).

Definition 5 (Conversion). *$s \equiv t$ is the smallest equivalence relation that contains $=_\alpha$ and $\to_\beta$ and is closed under all term constructors.*

To decide $s \equiv t$, the kernel normalizes both terms and compares the results with $=_\alpha$. This is correct because β-reduction is confluent: different reduction orders cannot lead to different normal forms. It also requires every well-typed term to *have* a normal form. That is true for the system in this section, but it is a deep theorem (strong normalization) [**?**]. A practical kernel also caps the number of reduction steps, so that a bug cannot make it loop forever.

6 UNIVERSES: WHY Sort u?

Types need types. Every Π and λ carries the type A of its variable, and the typing rules require A to be a type. To state "for every proposition P", we bind P with Π or λ, so we need a type for P itself: a type whose elements are propositions. That is a universe. In $\Pi(P{:}\mathrm{Sort}\ 0).\ P \to P$, the annotation Sort 0 says "P ranges over propositions".

Without universes, a proposition variable could only be an assumption placed in the context from outside:

$$P : \mathrm{Sort}\ 0 \vdash \lambda(h{:}P).\ h : P \to P \qquad \text{versus} \qquad \vdash \lambda(P{:}\mathrm{Sort}\ 0).\ \lambda(h{:}P).\ h : \Pi(P{:}\mathrm{Sort}\ 0).\ P \to P.$$

The left judgment proves $P \to P$ for one fixed P. The right one is a single closed term that proves it for every P, and it can be applied later to any particular proposition. The second form needs Sort 0 to be a well-typed term, so Sort 0 needs a type.

Type is not a Type. The simplest choice would be a single universe Type with Type : Type. This is inconsistent: Girard showed that it allows a closed term of every type, including $\Pi(P{:}\mathrm{Type}).\ P$, which corresponds to the proposition False [**? ?**]. The argument is a type-theoretic analogue of Russell's paradox about the set of all sets. A kernel with Type : Type would accept a proof of every statement.

A hierarchy. We avoid the paradox by never letting a universe contain itself. Instead, each universe lives in the next one:

$$\mathrm{Sort}\ 0 : \mathrm{Sort}\ 1 : \mathrm{Sort}\ 2 : \cdots$$

This is the rule T-Sort below. Each level exists to hold the level beneath it. In practice a few levels suffice; higher levels appear when one quantifies over large collections, such as the category of all small categories.

Where a Π type lives. A Π type lives in the larger of the universes of its domain and codomain (rule T-Pi). So $P \to P$ lives in Sort 0 when $P : \mathrm{Sort}\ 0$, but $\Pi(P{:}\mathrm{Sort}\ 0).\ P \to P$ lives in Sort 1, because it quantifies over all of Sort 0. A type that quantifies over a universe is never a member of that universe. This property is called *predicativity*.

Comparison with Lean. In Lean [**?**], Sort 0 is Prop and Sort $(u+1)$ is Type u. Prop has two extra features that our Sort 0 does not: *proof irrelevance* (any two proofs of the same proposition are equal) and *impredicativity* ($\forall\ (P : \mathrm{Prop}),\ P \to P$ is again in Prop, not one level up). Lean also has *universe polymorphism*, so definitions can be stated once for all levels u. In our system every level is a concrete number and every level follows the same rules.

7 TYPING RULES

Contexts and judgments. A *context* Γ is an ordered list of assumptions $x_1 : A_1, \ldots, x_n : A_n$. The order matters: A_i may mention $x_1, \ldots, x_{i-1}$, as in $P : \mathrm{Sort}\ 0, h : P$. We use two judgments:

- $\vdash \Gamma$ ok: the context Γ is well formed.
- $\Gamma \vdash t : A$: under the assumptions in Γ, the term t has type A.

Table 2. Design choices for universes.

Design	What can be quantified over	Drawback
No universes	values only	cannot say "for every proposition"
Type : Type	everything	inconsistent
Two sorts $* : \Box$	$*$ only	$\Box$ itself has no type
Sort u : Sort $(u+1)$	any level	levels must be tracked

$$\frac{}{\vdash \cdot \text{ ok}}\ (\text{C-Empty}) \qquad \frac{\vdash \Gamma \text{ ok} \qquad \Gamma \vdash A : \text{Sort } u \qquad x \notin \text{dom}(\Gamma)}{\vdash \Gamma, x : A \text{ ok}}\ (\text{C-Ext})$$

$$\frac{\vdash \Gamma \text{ ok}}{\Gamma \vdash \text{Sort } u : \text{Sort } (u+1)}\ (\text{T-Sort}) \qquad \frac{\vdash \Gamma \text{ ok} \qquad (x : A) \in \Gamma}{\Gamma \vdash x : A}\ (\text{T-Var})$$

$$\frac{\Gamma \vdash A : \text{Sort } u \qquad \Gamma, x : A \vdash B : \text{Sort } v}{\Gamma \vdash \Pi(x{:}A).\, B : \text{Sort } (\max(u,v))}\ (\text{T-Pi}) \qquad \frac{\Gamma \vdash A : \text{Sort } u \qquad \Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda(x{:}A).\, t : \Pi(x{:}A).\, B}\ (\text{T-Lam})$$

$$\frac{\Gamma \vdash f : \Pi(x{:}A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[x \mapsto a]}\ (\text{T-App}) \qquad \frac{\Gamma \vdash t : A \qquad \Gamma \vdash B : \text{Sort } u \qquad A \equiv B}{\Gamma \vdash t : B}\ (\text{T-Conv})$$

Fig. 2. Well-formed contexts and typing rules.

Read through Curry–Howard, $\Gamma \vdash t : A$ says that t is a proof of A from the hypotheses Γ.

We read each rule from the bottom up, as a recipe for checking a term.

C-Empty, C-Ext. A context is built one assumption at a time, and each new assumption must have a type as its type. The side condition $x \notin \text{dom}(\Gamma)$ forbids shadowing inside a context. Since we work up to α-equivalence, this is no restriction on terms: a binder that would shadow can always be renamed first. §10 shows why this condition matters.

T-Sort. Every universe lives in the next one (§6).

T-Var. A variable has the type it was assumed to have. Logically, this is "use a hypothesis".

T-Pi. $\Pi(x{:}A).\, B$ is a type if A is a type and B is a type once we assume $x : A$. It lives in the larger of the two universes. Logically, this rule says when $A \to B$ and $\forall x{:}A.\, B$ are well-formed propositions.

T-Lam. To build a function from A to B, assume $x : A$ and build $t : B$. Logically, this is the introduction rule for $\to$ and $\forall$: to prove $A \to B$, assume A and prove B. Note that B may mention x, and it is kept as is in the resulting Π type.

T-App. If f maps each $x : A$ to an element of B, and $a : A$, then $f\ a$ has type B with x replaced by a. Logically, this is modus ponens when B does not mention x, and instantiation of a universal statement when it does. For example, if $f : \Pi(n{:}\text{Nat}).\, n + 0 = n$, then $f\ 3 : 3 + 0 = 3$.

T-Conv. Types that are equal up to computation are interchangeable. This is the only rule that is not determined by the shape of t.

Why application matters. Without T-App, a hypothesis can be returned but never used. Roughly, the provable propositions are then those whose conclusion is literally one of the hypotheses, such as $P \to Q \to P$. Composition, $(P \to Q) \to (Q \to R) \to P \to R$, has the proof $\lambda(f{:}P \to Q).\, \lambda(g{:}Q \to R).\, \lambda(p{:}P).\, g\ (f\ p)$, which needs application.

8 A TYPE CHECKING ALGORITHM

The rules in Figure 2 define which judgments hold, but they are not yet an algorithm, because T-Conv can be applied at any point. The standard solution splits type checking into two mutually recursive functions [**?**]: Infer computes the type of a term, and Check verifies a term against an expected type. Conversion is used only where two types must be compared: in Check, and when a type must have a particular shape (a Π or a Sort).

Algorithm 1 Type inference and checking.

```
function Infer(Γ, t)
    if t = Sort u then return Sort (u + 1)
    else if t = x then
        if (x : A) ∈ Γ then return A
        else fail "unbound variable"
    else if t = Π(x:A). B then
        u ← InferSort(Γ, A)
        (x′, B′) ← Freshen(Γ, x, B)
        v ← InferSort((Γ, x′ : A), B′)
        return Sort (max(u, v))
    else if t = λ(x:A). s then
        InferSort(Γ, A)
        (x′, s′) ← Freshen(Γ, x, s)
        B ← Infer((Γ, x′ : A), s′)
        return Π(x′:A). B
    else if t = f a then
        T ← Normalize(Infer(Γ, f))
        if T = Π(x:A). B then
            Check(Γ, a, A)
            return B[x ↦ a]
        else fail "not a function"

function Check(Γ, t, T)
    T′ ← Infer(Γ, t)
    if Normalize(T′) =α Normalize(T) then return
    else fail "type mismatch"

function InferSort(Γ, A)
    T ← Normalize(Infer(Γ, A))
    if T = Sort u then return u
    else fail "expected a type"

function Freshen(Γ, x, b)
    if x ∈ dom(Γ) then
        choose x′ ∉ dom(Γ) ∪ FV(b)
        return (x′, b[x ↦ x′])
    else return (x, b)
```

Some remarks on Algorithm 1:

- Each case corresponds to exactly one rule of Figure 2, so the algorithm is *syntax-directed*. Finding a proof may be hard, but checking a given proof is mechanical.
- InferSort implements the premises of the form $\Gamma \vdash A : \text{Sort } u$. It is also how the kernel decides whether a term is a type at all: A is a type exactly when its type normalizes to a universe. A λ-abstraction can never be a type, because its inferred type is always a Π.

- Freshen renames a binder whose name is already in the context, implementing the side condition of C-Ext.
- Starting from the empty context, every context the algorithm builds is well formed, because each assumption is added only after InferSort succeeds. A caller that starts from a non-empty context must check that context separately.
- Termination depends on Normalize terminating, which holds for well-typed terms by strong normalization (§5).

9 WORKED EXAMPLES

The identity proof. Let $\Gamma_1 = P : \mathrm{Sort}\ 0$ and $\Gamma_2 = \Gamma_1, h : P$. Omitting the well-formedness premises $\vdash \Gamma$ ok, the derivation of the opening example is

$$\dfrac{\dfrac{}{\cdot \vdash \mathrm{Sort}\ 0 : \mathrm{Sort}\ 1}\ \textsc{T-Sort} \qquad \dfrac{\dfrac{}{\Gamma_1 \vdash P : \mathrm{Sort}\ 0}\ \textsc{T-Var} \qquad \dfrac{}{\Gamma_2 \vdash h : P}\ \textsc{T-Var}}{\Gamma_1 \vdash \lambda(h{:}P).\, h : P \to P}\ \textsc{T-Lam}}{\cdot \vdash \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P}\ \textsc{T-Lam}$$

Algorithm 1 produces the type $\Pi(P{:}\mathrm{Sort}\ 0).\, \Pi(h{:}P).\, P$. It is α-equivalent to the stated type, because $P \to P$ abbreviates $\Pi(_{:}P).\, P$ and the names h and _ do not occur in the body.

A wrong proof is rejected. For $\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, P$, the body P has type $\mathrm{Sort}\ 0$, not P, so Check reports a type mismatch.

Using a theorem. Let $id = \lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(h{:}P).\, h$ and $\Gamma = Q : \mathrm{Sort}\ 0$. By T-App,

$$\dfrac{\Gamma \vdash id : \Pi(P{:}\mathrm{Sort}\ 0).\, P \to P \qquad \Gamma \vdash Q : \mathrm{Sort}\ 0}{\Gamma \vdash id\ Q : (P \to P)[P \mapsto Q]}\ \textsc{T-App}$$

and $(P \to P)[P \mapsto Q] = Q \to Q$. Applying a proof of a universal statement to a particular proposition yields a proof about that proposition, and substitution computes what is being proved.

An exercise. Prove $\Pi(P{:}\mathrm{Sort}\ 0).\, \Pi(Q{:}\mathrm{Sort}\ 0).\, P \to Q \to P$. One answer is

$$\lambda(P{:}\mathrm{Sort}\ 0).\, \lambda(Q{:}\mathrm{Sort}\ 0).\, \lambda(p{:}P).\, \lambda(q{:}Q).\, p.$$

The proof never uses q; this system allows hypotheses to go unused.

10 A PITFALL: VARIABLE CAPTURE IN CONTEXTS

It is tempting to drop the side condition $x \notin \mathrm{dom}(\Gamma)$ and let a new assumption simply shadow an old one, as most programming languages do. Doing so makes the kernel unsound. Consider

$$t = \lambda(x{:}\mathrm{Sort}\ 0).\, \lambda(y{:}x).\, \lambda(x{:}\mathrm{Sort}\ 0).\, y.$$

The variable y has the *outer* x as its type, and the innermost λ ignores its argument. So the correct type of t is

$$\Pi(x{:}\mathrm{Sort}\ 0).\, \Pi(y{:}x).\, \Pi(x'{:}\mathrm{Sort}\ 0).\, x.$$

A checker without Freshen looks up y in the context $x : \mathrm{Sort}\ 0, y : x, x : \mathrm{Sort}\ 0$ and returns its type, the bare name x. At that point x refers to the inner binder, and the checker returns

$$\Pi(x{:}\mathrm{Sort}\ 0).\, \Pi(y{:}x).\, \Pi(x{:}\mathrm{Sort}\ 0).\, x,$$

which says "from any element of any type, produce an element of every type". Given $A : \mathrm{Sort}\ 0$ and $a : A$, the term $t\ A\ a$ then has type $\Pi(B{:}\mathrm{Sort}\ 0).\, B$, a proof of False. Any extension of the system with an inhabited base type, such as Nat with $0 : \mathrm{Nat}$, provides such A and a in the empty context.

The problem is the same as in substitution (§3): a name is moved into a scope where it means something else. It affects any representation that identifies variables by name, whether the context is a list or a hash map. There are three common remedies:

(1) Reject shadowing outright. This is the simplest option, but the reserved name _ used by $A \to B$ must be exempt and never added to the context.
(2) Rename on entry, as Freshen does.
(3) Represent bound variables by position (de Bruijn indices [**?**]) or use the *locally nameless* style, in which bound variables are indices and free variables carry unique identifiers. Lean 4 takes the latter approach.

A test suite should include t above and check that the kernel either rejects it or assigns it the correct type.

11 PROPERTIES

We state, without proof, the standard properties of this system. It is an instance of a Pure Type System with a predicative hierarchy, and the proofs can be found in Barendregt [**?**].

Theorem 1 (Uniqueness of types). *If* $\Gamma \vdash t : A$ *and* $\Gamma \vdash t : B$*, then* $A \equiv B$*.*

Theorem 2 (Type correctness). *If* $\Gamma \vdash t : A$*, then either* $A = \mathsf{Sort}\ u$ *for some* u*, or* $\Gamma \vdash A : \mathsf{Sort}\ u$ *for some* u*.*

Theorem 3 (Subject reduction). *If* $\Gamma \vdash t : A$ *and* $t \to_\beta t'$*, then* $\Gamma \vdash t' : A$*.*

Theorem 4 (Strong normalization). *If* $\Gamma \vdash t : A$*, then every sequence of* β*-reductions starting from* t *is finite.*

Together these give the two guarantees we want from a kernel. *Decidability*: Algorithm 1 terminates on every input, so whether a proof is correct can always be determined. *Consistency*: there is no closed term of type $\Pi(P{:}\mathsf{Sort}\ 0).\ P$. By strong normalization and subject reduction, such a term would have a normal form of the same type. That normal form must be $\lambda(P{:}\mathsf{Sort}\ 0).\ s$ with $P : \mathsf{Sort}\ 0 \vdash s : P$ and s normal. A normal term is a universe, a Π, a λ, or a variable applied to arguments, and none of these can have type P in that context: the only variable available is P itself, whose type is $\mathsf{Sort}\ 0$, not P.